PROJECT INFI / ARIS

The Voss Binary

Formal Specification

A Governed Runtime Calculus for the ARIS Cognitive Operating System

Jon Halstead

Version 1.0 | May 2026

CLASSIFICATION: FORMAL SPECIFICATION — NORMATIVE

Contents

§0	Abstract
§1	Introduction
§2	Binary Layout
§3	Instruction Set Architecture (ISA)
§4	VM State Model
§5	Instruction Semantics
§6	GRE Pipeline
§7	The Seven Λ Laws
§8	The Five Δ Axioms
§9	Canonical Invariants
§10	REP Trace Format
§11	Sample Program and Execution Trace
§12	Verifier Logic
§13	Execution Model
§14	System Guarantees
A	Appendix A — Quick Reference Card

§0 — Abstract

The Voss Binary is a 16-bit fixed-width instruction set architecture and governed runtime calculus designed for the ARIS Cognitive Operating System. It encodes the seven Λ (Lambda) laws and five Δ (Delta) axioms directly into the instruction path, enforced by a deterministic 9-stage GRE (Governed Runtime Engine) pipeline. This specification defines the binary layout, instruction semantics, invariant system, REP trace format, and formal verification method. The Voss Binding ensures that no system evolves in isolation — all state must bind to influence, and that binding carries cost into all future cycles.

§1 — Introduction

The Voss Binary is the machine-level realization of the Voss Binding (Λ) — a governed runtime calculus in which laws are not policies, configurations, or advisory constraints, but structural machine code. Where conventional systems enforce governance through external policy layers, the Voss Binary compiles governance directly into instruction decoding and state transitions. The coupling is unbreakable without recompiling the binary itself.

The GRE (Governed Runtime Engine) pipeline sits in the hot path of every execution cycle. The seven Λ laws govern binding and fate. The five Δ axioms govern debt and delta accumulation. Both are compiled directly into instruction classes and enforced at decode time. No instruction may execute without traversing the full pipeline, and no pipeline stage may be bypassed or reordered.

> “
>
> *No system evolves in isolation. After stabilization, all state must bind to influence, and that binding must carry cost into all future cycles.*
>
> ”

§1.1 — Classification

Domain	MetaDomain
Type	Cycle Boundary Operator
Position	Post-Δ, Pre-next `1000`

§1.2 — Formal Placement

The Voss Binding occupies the terminal boundary position in the ARIS cycle sequence. The canonical cycle flow is expressed in both binary notation and symbolic form:

```
Binary:   0001 → 1000 → 1001 → 1010 → 1111 → 1001 → 0001' → Δ → Λ → next 1000
Symbolic: S₀   →  C   →  L₁  →  M   →  D   →  L₂  →  S₁   → Δ → Λ → next C
```

The Λ operator executes after all Δ axioms have been evaluated and settled. It marks the formal boundary between the current cycle and the next, ensuring that no cycle transition occurs without binding enforcement.

§2 — Binary Layout

All Voss instructions are encoded as fixed-width 16-bit instruction words. There are no variable-length encodings, no instruction prefixes, and no alignment exceptions. The fixed width guarantees constant-time fetch and deterministic decode.

```
15  14  13  12  11  10   9   8   7   6   5   4   3   2   1   0
┌───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┐  │
CLASS     │      OPCODE      │           IMM / FLAGS           │  │    [15:12]
│     [11:8]       │            [7:0]              │
└───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┘         4
bits           4 bits                  8 bits
```

- **Bits [15:12] — CLASS** (4 bits): High-level instruction category. Determines which pipeline stages are active and which governance checks apply.
- **Bits [11:8] — OPCODE** (4 bits): Specific operation within the class. Combined with CLASS, this forms the full 8-bit operation identifier.
- **Bits [7:0] — IMM/FLAGS** (8 bits): Small immediate value, flag field, or micro-selector. Interpretation is OPCODE-dependent.

Table 1: Class Map

Code (Binary)	Hex	Class Name	Description
`0000`	`0x0`	**FLOW**	Cycle control: `NEXT`, `ADMIT`, `WAIT`, `HALT`, `FAULT`
`0001`	`0x1`	**BIND/Λ**	Lambda operations: `BIND`, `UNBIND`, `LOCK`, `RELEASE`, `Λ_CHAIN`
`0010`	`0x2`	**DELTA/Δ**	Delta operations: `Δ_ADD`, `Δ_SETTLE`, `Δ_DEGRADE`, `Δ_PROPAGATE`
`0011`	`0x3`	**INVARIANT**	Assertion: `ASSERT_INV`, `GUARD_INV`, `FAIL_INV`
`0100`	`0x4`	**META/GRE**	Tracing and hooks: `TRACE`, `GRE_MARK`, `GRE_NEXT_K`
`0101–1111`	`0x5–0xF`	**RESERVED**	Reserved for future extension

§3 — Instruction Set Architecture (ISA)

The complete Voss ISA comprises 20 instructions organized across five classes. All encodings are normative. Implementations MUST reject any instruction word whose CLASS or OPCODE falls outside the defined set.

Table 2: Complete Voss ISA

FLOW Class (CLASS = `0x0`)

Binary Encoding	Hex	Mnemonic	Operands	Description
`0000 0000 iiiiiiii`	`0x00ii`	`NEXT i8`	i: step count	Advance GRE cycle by i steps
`0000 0001 ffffffff`	`0x01ff`	`ADMIT f8`	f: context flags	Mark context admitted under flags f
`0000 0010 ffffffff`	`0x02ff`	`WAIT f8`	f: wait flags	Suspend until condition f met
`0000 1111 00000000`	`0x0F00`	`HALT`	—	Controlled termination
`0000 1111`	`0x0F01`	`FAULT`	—	Fault state

Binary Encoding	Hex	Mnemonic	Operands	Description
`00000001`				entry

BIND/Λ Class (CLASS = `0x1`)

Binary Encoding	Hex	Mnemonic	Operands	Description
`0001 0000 ttttffff`	`0x10tf`	`BIND t4,f4`	t: target, f: fate flags	Bind target t under fate flags f
`0001 0001 ttttffff`	`0x11tf`	`UNBIND t4,f4`	t: target, f: fate flags	Remove fate flags f from target t
`0001 0010 ttttffff`	`0x12tf`	`LOCK t4,f4`	t: target, f: fate flags	Irreversibly lock fate flags on target t
`0001 0011 ttttffff`	`0x13tf`	`RELEASE t4,f4`	t: target, f: fate flags	Release (non-locked) fate flags from target t
`0001 0100 ccccffff`	`0x14cf`	`Λ_CHAIN c4,f4`	c: chain ID, f: flags	Apply fate flags to all targets in chain c

DELTA/Δ Class (CLASS = `0x2`)

Binary Encoding	Hex	Mnemonic	Operands	Description
`0010 0000 ttttdddd`	`0x20td`	`Δ_ADD t4,d4`	t: target, d: delta	Add delta d to target t
`0010 0001 ttttdddd`	`0x21td`	`Δ_SETTLE t4,d4`	t: target, d: delta	Settle (subtract) delta d from target t
`0010 0010 ttttdddd`	`0x22td`	`Δ_DEGRADE t4,d4`	t: target, d: delta	Apply degradation model to target t
`0010 0011 ttttdddd`	`0x23td`	`Δ_PROPAGATE t4,d4`	t: target, d: delta	Propagate delta from t to coupled targets

INVARIANT Class (CLASS = `0x3`)

Binary Encoding	Hex	Mnemonic	Operands	Description
`0011 0000 iiiiiiii`	`0x30ii`	`ASSERT_INV i8`	i: invariant ID	Assert invariant i; FAULT if false
`0011 0001 iiiiiiii`	`0x31ii`	`GUARD_INV i8`	i: invariant ID	Guard invariant i; WAIT if false
`0011 1111 iiiiiiii`	`0x3Fii`	`FAIL_INV i8`	i: invariant ID	Unconditional invariant failure

META/GRE Class (CLASS = `0x4`)

Binary Encoding	Hex	Mnemonic	Operands	Description
`0100 0000 mmmmmmmm`	`0x40mm`	`TRACE m8`	m: tag	Emit REP trace record tagged with m
`0100 0001 mmmmmmmm`	`0x41mm`	`GRE_MARK m8`	m: marker	Mark structural boundary (phase/module)
`0100 0010 kkkkkkkk`	`0x42kk`	`GRE_NEXT_K k8`	k: cycle count	Logical cycle jump by k (no PC change)

§4 — VM State Model

The Voss VM state is a deterministic, fully-observable structure. All state is inspectable at any cycle boundary. There is no hidden state, no speculative execution, and no out-of-order reordering.

Formal State Definition:

```
VossState = {     pc          : u16               -- program counter
cycle       : u32               -- GRE cycle counter     status      : OK |
WAIT | FAULT | HALT      delta[t]    : int               -- Δ debt table (per
target)     fate[t]     : u8                -- Λ fate flags (per target)
locked[t]   : u8                -- Λ locked bits (per target)      inv[i]
: Predicate         -- invariant definitions     inv_ok      : bool
-- last invariant result      ctx.flags       : u8            -- admit/wait
flags     ctx.wait_flags  : u8            -- recorded wait condition
ctx.wait_reason : WaitReason    -- reason for WAIT state }
```

§4.1 — Registers

The Voss VM exposes three core registers. All other state is table-indexed (keyed by target ID or invariant ID).

Register	Type	Description
`pc`	`u16`	Program counter. Points to the next instruction word in the code segment. Incremented by 1 during FETCH (stage S0). Range: `0x0000`–`0xFFFF`.
`cycle`	`u32`	GRE cycle counter. Monotonically non-decreasing. Incremented by `NEXT` and `GRE_NEXT_K` instructions. Used for temporal ordering in REP traces.
`status`	enum	Execution status. Legal values: `OK`, `WAIT`, `FAULT`, `HALT`. Only transitions via legal instructions are permitted (see §9, INV_6).

§5 — Instruction Semantics

This section defines the precise execution semantics of every instruction in the Voss ISA. Each definition is normative. An implementation that deviates from these semantics is non-conformant.

§5.1 — FLOW Semantics

Instruction	Execution Semantics
`NEXT i`	`pc` is already advanced by fetch. `cycle += i`. Status unchanged if `OK`. If `status != OK`, behavior is undefined (implementations SHOULD fault).
`ADMIT f`	`ctx.flags |= f`. If any invariant forbids admission under the resulting flag set, `status = FAULT`. Otherwise, status unchanged.
`WAIT f`	`status = WAIT`. `ctx.wait_flags = f`.

Instruction	Execution Semantics
	Execution suspends. The external scheduler must clear the wait condition to resume.
`HALT`	`status = HALT`. Execution terminates. No further instructions are fetched.
`FAULT`	`status = FAULT`. Enters fault state. Execution terminates unless a fault handler is registered.

§5.2 — BIND/Λ Semantics

Instruction	Execution Semantics
`BIND t,f`	`fate[t] |= f`. If `f` contradicts existing fate configuration for target `t` → `status = FAULT`. Coupling debt: `debt.coupling += 5` (standard bind); `debt.coupling += 10` if target was already bound (re-bind penalty).
`UNBIND t,f`	If `(locked[t] & f) != 0` → `status = FAULT`. Else `fate[t] &= ~f`.
`LOCK t,f`	`locked[t] |= f`; `fate[t] |= f`. The lock is irreversible. Once set, these bits can never be cleared for the lifetime of the execution.
`RELEASE t,f`	If `(locked[t] & f) != 0` → `status = FAULT`. Else `fate[t] &= ~f`.
`Λ_CHAIN c,f`	For all targets `x` in chain `c`: `fate[x] |= f`. If any member diverges from shared fate after application → invariant failure → `status = FAULT`.

§5.3 — DELTA/Δ Semantics

Instruction	Execution Semantics
`Δ_ADD t,d`	`delta[t] += d`. If overflow occurs → `status = FAULT`.
`Δ_SETTLE t,d`	If `delta[t] < d` → `status = FAULT` (non-negativity violation). Else `delta[t] -= d`.
`Δ_DEGRADE t,d`	`delta[t] = max(0, delta[t] - d)`. May trigger degradation flags on target `t` if delta reaches zero.
`Δ_PROPAGATE t,d`	For each `u` in `coupled(t)`: `delta[u] += f(d)`

Instruction	Execution Semantics
	where f is the coupling transfer function. If any invariant is violated during propagation → `status = FAULT`. Uncoupled propagation is illegal.

§5.4 — INVARIANT Semantics

Instruction	Execution Semantics
`ASSERT_INV i`	Evaluate `inv[i]`. If false → `status = FAULT`. Set `inv_ok = result`.
`GUARD_INV i`	Evaluate `inv[i]`. If false → `status = WAIT`; `ctx.wait_reason = INV_FAIL(i)`. Execution suspends until the invariant condition is restored.
`FAIL_INV i`	`status = FAULT`; `fault_reason = INV_EXPLICIT(i)`. Unconditional — the invariant is not evaluated; failure is asserted directly.

§5.5 — META/GRE Semantics

Instruction	Execution Semantics
`TRACE m`	Emit a REP trace record tagged with marker m. No state change. The trace record captures the full VM state snapshot at this point in the pipeline.
`GRE_MARK m`	Mark a structural boundary (phase transition or module boundary). `mark_stack.push((cycle, m))`. Used for post-hoc structural analysis of execution traces.
`GRE_NEXT_K k`	`cycle += k`. Does not change `pc`. This is a logical cycle advance without instruction fetch, used for cycle-alignment across parallel GRE instances.

§6 — GRE Pipeline

The Governed Runtime Engine — 9-Stage Pipeline

Every instruction in the Voss ISA traverses the 9-stage GRE pipeline. The pipeline is strictly sequential and deterministic. There is no out-of-order execution, no speculative branching, and no pipeline bypass. Some instructions skip certain stages (e.g., a FLOW instruction has no effect in stages S3/S4), but the pipeline structure is always present and the stage ordering is invariant. This is the deterministic hot path.

Table 3: GRE Pipeline Stages

Stage	ID	Name	Function
0	S0	**FETCH**	Load next 16-bit instruction word from `code[pc]`. Increment `pc`.
1	S1	**DECODE**	Split instruction into CLASS [15:12], OPCODE [11:8], IMM/FLAGS [7:0]. Validate CLASS and OPCODE against known set.
2	S2	**PRECHECK**	Evaluate pre-execution invariants. Check lock conflicts. Validate domain permissions. FAULT if any precondition fails.
3	S3	**APPLY_Δ**	Apply delta changes if instruction is DELTA class. Compute new `delta[t]` values. Enforce Δ axioms.
4	S4	**APPLY_Λ**	Commit binding/fate changes if instruction is BIND class. Update `fate[t]` and `locked[t]`. Enforce Λ laws.
5	S5	**FLOW_CTRL**	Process admit/wait/next/branch decisions. Update `cycle`, `status`, and `ctx` fields as required.

Stage	ID	Name	Function
6	S6	**DEGRADE**	Apply degradation model if required. Evaluate time-decay and entropy functions on delta entries.
7	S7	**EMIT/TRACE**	Optional logging. REP hooks. Trace emission. State snapshot capture for verifier consumption.
8	S8	**HALT/FAULT**	Terminal or recoverable fault handling. If `status == HALT` or `status == FAULT`, pipeline terminates.

```
Pipeline Flow:    S0       S1        S2         S3         S4         S5
S6        S7          S8 FETCH → DECODE → PRECHECK → APPLY_Δ → APPLY_Λ →
FLOW_CTRL → DEGRADE → EMIT/TRACE → HALT/FAULT   │                   │
│          │          │                        │           │   └─ code[pc]
└─ INV     └─ delta   └─ fate   └─ cycle/status       └─ REP       └─ TERM
```

§7 — The Seven Λ Laws

The seven Λ laws are structural invariants of the Voss Binary. They are not policy directives or runtime configuration — they are compiled into the instruction decoder and pipeline stages. Violation of any Λ law results in an immediate `FAULT`.

Λ_1 — Origin Law (No Spontaneous Fate)

Any fate bit set on a target must originate from a Λ-class instruction. If `fate[t]` changes between cycles, then `instr.class` must be `BIND/Λ`. Fate does not appear spontaneously, and no non-Λ instruction may modify the fate table.

Formal: ∀t, ∀n: `fate[t]n+1 ≠ fate[t]n` ⇒ `instrn.class == BIND/Λ`

Λ_2 — Monotone Lock Law (Irreversible Lock)

Once a fate bit is locked, it can never be cleared. The lock operation is monotone and irreversible.

Formal: ∀t, ∀b: if `(locked[t] & b) != 0` at cycle n, then `(fate[t] & b) != 0` for all k ≥ n

Λ_3 — Legal Release Law (No Unlock of Locked Fate)

A `RELEASE` or `UNBIND` instruction may only clear bits that are not locked. Any attempt to clear a locked bit is a structural violation.

Formal: if `instr == RELEASE/UNBIND t,f`, then `(locked[t] & f) == 0`

Λ_4 — Coupling Law (Shared Fate in Chains)

All members of a Λ chain share the same fate bits. Chain formation creates a coupling constraint that persists until explicit dissolution.

Formal: ∀ chain C, ∀t,u ∈ C: `fate[t] == fate[u]` at every cycle after chain formation

Λ_5 — Chain Persistence Law (No Silent Decoupling)

Once a Λ chain is formed, its membership cannot silently shrink. A target leaves a chain only via an explicit Λ instruction that faults or rebinds according to the specification. No implicit decoupling, garbage collection, or timeout-based removal is permitted.

Λ_6 — Status-Fate Coherence Law

Certain fate configurations imply legal status values only. If `fate[t]` encodes

“terminally bound,” then status may only be `HALT` or `OK`, never `WAIT` with contradictory flags. The fate-status relationship is a bijection within defined equivalence classes.

Λ_7 — Invariant Alignment Law

All Λ operations must respect invariant checks. Any Λ instruction that would violate an invariant must result in `status = FAULT` in the same cycle. Deferred invariant failure is not permitted for Λ-class instructions.

§8 — The Five Δ Axioms

The five Δ axioms govern all delta (debt) accumulation, settlement, and propagation within the Voss Binary. They are structural invariants checked at every cycle boundary.

Δ_1 — Non-Negativity

No Δ entry may go negative. *Formal:* ∀t: `delta[t] ≥ 0`

Δ_2 — Locality

Δ may only change via Δ-class instructions. Between cycles n and n+1, if `instr.class != DELTA`, then `delta[t]n == delta[t]n+1` for all t.

Δ_3 — Conservation

For `Δ_ADD`, `Δ_SETTLE`, `Δ_DEGRADE`, and `Δ_PROPAGATE` — the expected `delta'` is recomputable from `prev_delta` and the instruction operands. Any mismatch between the computed and observed delta is an illegal transition and

constitutes a verification failure.

Δ_4 — Propagation Integrity

`Δ_PROPAGATE` must distribute delta only to coupled targets (via Λ chains). Any propagation to an uncoupled target is a `FAULT`. The coupling graph is defined by active Λ chains at the time of propagation.

Δ_5 — Coupling Cost

The Voss Binding always increases coupling debt. Binding is never free.

- `debt.coupling += 5` for standard bind
- `debt.coupling += 10` for re-binding an already-bound target (double penalty)

§9 — Canonical Invariants

The following invariants constitute the canonical invariant set. Conformant implementations MUST check these invariants at the specified points. Additional implementation-defined invariants MAY be added but MUST NOT weaken or override the canonical set.

Table 4: Canonical Invariant Set

ID	Name	Predicate	Checked After
`INV_1`	Δ Non-Negativity	∀t: `delta[t] ≥ 0`	`Δ_SETTLE`, `Δ_DEGRADE`, `Δ_PROPAGATE`
`INV_2`	Δ Locality	If `instr.class != DELTA` then delta unchanged	Every cycle
`INV_3`	Λ Lock Integrity	Locked fate bits never cleared	Every cycle

ID	Name	Predicate	Checked After
`INV_4`	Λ Coupling Consistency	All chain members share same fate bits	Every cycle
`INV_5`	Cycle Monotonicity	`cyclen+1 ≥ cyclen`; if `status == OK` then strictly increasing	Every cycle
`INV_6`	Status Discipline	Status transitions only via legal instructions	Every cycle
`INV_7`	Admit/Wait Coherence	Cannot be admitted and waiting with contradictory flags	`ADMIT`, `WAIT`

§10 — REP Trace Format

The REP (Runtime Execution Protocol) trace is the canonical observability format for the Voss Binary. Every execution produces a REP trace. The trace is the sole input to the verifier (§12) and the authoritative record of execution history.

§10.1 — JSON-Line Format

Each REP trace record is a self-contained JSON object emitted on a single line. The canonical structure is as follows:

```
{   "cycle": 42,   "pc": 12,   "instr": {     "raw": "0x2011",     "class": "DELTA",     "opcode": "DELTA_ADD",     "imm": 17   },   "state": {     "status": "OK",     "delta": { "1": 3, "2": 0 },     "fate": { "1": "0x03", "2": "0x00" },     "locked": { "1": "0x01" }   },   "invariants": { "checked": [17],     "ok": true   },   "events": [     "DELTA_ADD t=1 d=1", "COUPLING_OK",     "CYCLE_ADVANCE"   ] }
```

§10.2 — Field Definitions

Field	Type	Definition
`cycle`	`u32`	GRE cycle index at the time of instruction execution.
`pc`	`u16`	Program counter value at the start of fetch (before increment).
`instr.raw`	`hex`	Raw 16-bit instruction word in hexadecimal.
`instr.class`	`string`	Decoded class name (FLOW, BIND, DELTA,

Field	Type	Definition
		INVARIANT, META).
`instr.opcode`	`string`	Decoded opcode mnemonic.
`instr.imm`	`u8`	Immediate/flags field, decoded as unsigned integer.
`state.status`	`string`	VM status after instruction execution: OK, WAIT, FAULT, or HALT.
`state.delta`	`map`	Snapshot of the delta table (target ID → delta value).
`state.fate`	`map`	Snapshot of the fate table (target ID → fate flags as hex).
`state.locked`	`map`	Snapshot of the locked table (target ID → locked bits as hex).
`invariants.checked`	`array`	List of invariant IDs evaluated during this cycle.
`invariants.ok`	`bool`	True if all checked invariants passed; false otherwise.
`events`	`array`	Human-readable event tags describing state changes and governance decisions.

§10.3 — Compact Binary REP Format

For high-throughput or embedded scenarios, the compact binary REP format provides a fixed-width record suitable for streaming and fast diffing:

```
Offset  Size   Field ──────  ─────  ─────────────── 0x00    4      cycle
: u32    (GRE cycle counter) 0x04    2      pc          : u16    (program
counter at fetch) 0x06    1      class       : u8     (instruction class)
0x07    1      opcode      : u8     (instruction opcode) 0x08    1      imm
: u8     (immediate / flags) 0x09    1      status      : u8     (0=OK,
1=WAIT, 2=FAULT, 3=HALT) 0x0A    2      inv_mask    : u16    (bitmask of
checked invariants) 0x0C    4      delta_hash  : u32    (hash of delta table
snapshot) 0x10    4      fate_hash   : u32    (hash of fate table snapshot)
──────  ─────  ─────────────── Total: 20 bytes per record
```

The `delta_hash` and `fate_hash` fields use a deterministic hash function (implementation-defined, but MUST be consistent across a single execution trace). These hashes allow fast diffing across runs without transmitting full state snapshots.

§11 — Sample Program and Execution Trace

§11.1 — Golden Path Program

The following is the canonical ARIS/Voss test program — the “golden path” lifecycle flow. This program exercises every instruction class and demonstrates a complete bind-accumulate-settle-degrade-halt cycle.

```
Address  Hex     Instruction          Comment ────────  ───────
────────────────────  ──────────────────────────────────── 0x00     0x4001
TRACE 1              Start marker 0x01     0x0001  NEXT 1
Advance GRE cycle 0x02     0x2011  Δ_ADD t=1, d=1       Accumulate delta on
target 1 0x03     0x2011  Δ_ADD t=1, d=1       Accumulate delta on target 1
0x04     0x2111  Δ_SETTLE t=1, d=1    Settle delta on target 1 0x05
0x1011  BIND t=1, f=1        Bind target 1 under fate flag 0x1 0x06
0x3011  ASSERT_INV 0x11      Assert invariant 0x11 0x07     0x0111  ADMIT
f=0x11         Admit context under flags 0x11 0x08     0x0211  WAIT f=0x11
Wait for condition 0x11 0x09     0x0001  NEXT 1               Advance GRE
cycle (after resume) 0x0A     0x2211  Δ_DEGRADE t=1, d=1   Degrade target 1
0x0B     0x3012  ASSERT_INV 0x12      Post-degrade invariant assertion 0x0C
0x4002  TRACE 2              End marker 0x0D     0x0F00  HALT
Clean termination
```

§11.2 — Execution Trace

The following table traces the golden path program cycle by cycle, showing complete state evolution. Initial state: `cycle=0`, `pc=0x00`, `status=OK`, `delta[1]=0`, `fate[1]=0x00`.

Cycle	PC	Hex	Class	Operation	Status	Δ[1]	Fate[1]	Notes
0	0x00	`0x4001`	META	TRACE 1	OK	0	0x00	Start marker emitted to REP
0	0x01	`0x0001`	FLOW	NEXT 1	OK	0	0x00	cycle advances 0 → 1
1	0x02	`0x2011`	DELTA	Δ_ADD t=1,d=1	OK	1	0x00	delta[1]: 0 → 1
1	0x03	`0x2011`	DELTA	Δ_ADD t=1,d=1	OK	2	0x00	delta[1]: 1 → 2
1	0x04	`0x2111`	DELTA	Δ_SETTLE	OK	1	0x00	delta[1]: 2 → 1;

Cycle	PC	Hex	Class	Operation	Status	Δ[1]	Fate[1]	Notes
				t=1,d=1				INV_1 checked
1	0x05	`0x1011`	BIND	BIND t=1,f=1	OK	1	0x01	fate[1]: 0x00 → 0x01; debt += 5
1	0x06	`0x3011`	INVAR	ASSERT_INV 0x11	OK	1	0x01	Invariant 0x11 passes; inv_ok = true
1	0x07	`0x0111`	FLOW	ADMIT f=0x11	OK	1	0x01	Context admitted; ctx.flags |= 0x11
1	0x08	`0x0211`	FLOW	WAIT f=0x11	WAIT	1	0x01	Suspended; wait_flags = 0x11
1	0x09	`0x0001`	FLOW	NEXT 1	OK	1	0x01	Resumed by scheduler; cycle 1 → 2
2	0x0A	`0x2211`	DELTA	Δ_DEGRADE t=1,d=1	OK	0	0x01	delta[1]: max(0, 1−1) = 0
2	0x0B	`0x3012`	INVAR	ASSERT_INV 0x12	OK	0	0x01	Post-degrade invariant passes
2	0x0C	`0x4002`	META	TRACE 2	OK	0	0x01	End marker emitted to REP
2	0x0D	`0x0F00`	FLOW	HALT	HALT	0	0x01	Clean termination. All invariants

Cycle	PC	Hex	Class	Operation	Status	Δ[1]	Fate[1]	Notes
								satisfied.

Final State: `status = HALT`, `cycle = 2`, `pc = 0x0E`, `delta[1] = 0`, `fate[1] = 0x01`, `locked[1] = 0x00`. All seven Λ laws and five Δ axioms satisfied throughout execution.

§12 — Verifier Logic

§12.1 — Verification Method

The Voss verifier operates over the REP trace as its sole input. It does not inspect the binary, the VM implementation, or any internal state — only the sequence of REP records produced by execution. This is the **REP law-proof method**: if the trace is consistent with all laws and axioms, then the execution was governed. If any record violates a law or axiom, the execution is non-conformant.

The verifier maintains the following derived state across records:

- `prev_cycle`, `prev_pc` — previous cycle and program counter
- `prev_delta`, `prev_fate`, `prev_locked` — previous state snapshots
- `prev_status` — previous execution status
- `locked_seen` — cumulative set of all bits ever locked (per target)
- `chains` — active chain membership table

§12.2 — Verification Checks

Law / Axiom	Verification Check
Δ Non-Negativity (Δ_1)	For every record, assert `delta[t] ≥ 0` for all targets t present in the state snapshot.
Δ Locality (Δ_2)	If `instr.class != DELTA`, assert `state.delta == prev_delta`. Any delta change by a non-DELTA instruction is a violation.
Δ Conservation (Δ_3)	Recompute expected `delta'` from

Law / Axiom	Verification Check
	`prev_delta` and the instruction operands. Compare against the observed `state.delta`. Any mismatch is a violation.
Λ Lock Integrity (Λ_2)	When a bit enters `locked[t]`, record it in `locked_seen[t]`. For all subsequent records, verify that `(fate[t] & locked_seen[t]) == locked_seen[t]`. Any cleared locked bit is a violation.
Λ Coupling Consistency (Λ_4)	For each active chain, verify that all member targets share identical fate bits: `∀t,u ∈ chain: fate[t] == fate[u]`.
Status Discipline (INV_6)	Verify that each status transition was caused by a legal instruction. Map each `(prev_status, status)` pair to the instruction that caused it and confirm legality.
Invariant Alignment (Λ_7)	If `invariants.ok == false` in any record, verify that `status == FAULT` in the same record. Deferred failure is a violation.

§12.3 — Verdict

FORMAL VERDICT CONDITION

If all invariants hold for every record, and all derived checks over Δ, Λ, cycle, and status pass, then **for this execution**, the seven Λ laws and five Δ axioms held at the binary, instruction, and GRE-cycle level, as evidenced by the REP trace. The execution is **CONFORMANT**. Otherwise, the first failing record identifies the violation point and the execution is **NON-CONFORMANT**.

§13 — Execution Model

§13.1 — VM Core Loop

The canonical Voss VM execution loop follows a strict fetch-decode-execute pattern with the GRE pipeline enforced at every iteration:

```
void voss_run(VossState *vm, uint16_t *code, size_t len) {     while (vm-
>status == STATUS_OK && vm->pc < len) {         uint16_t word   = code[vm-
```

```
>pc++];           // S0: FETCH         uint8_t  cls    = (word >> 12) & 0x0F;
// S1: DECODE         uint8_t  opcode = (word >>  8) & 0x0F;         uint8_t
imm    = word & 0xFF;
precheck(vm);                                 // S2: PRECHECK          switch
(cls) {             case 0x0:  exec_flow(vm, opcode, imm);   break;  // S5:
FLOW_CTRL             case 0x1:  exec_bind(vm, opcode, imm);   break;  // S4:
APPLY_Λ             case 0x2:  exec_delta(vm, opcode, imm);  break;  // S3:
APPLY_Δ             case 0x3:  exec_invar(vm, opcode, imm);  break;  // S2:
INVARIANT             case 0x4:  exec_meta(vm, opcode, imm);   break;  // S7:
META/GRE            default:   vm->status = STATUS_FAULT;    break;  //
RESERVED         }          degrade(vm);                                 //
S6: DEGRADE         emit_trace(vm, word);                        // S7:
EMIT/TRACE         check_halt_fault(vm);                        // S8:
HALT/FAULT          vm->cycle++;     } }
```

§13.2 — Packaging Targets

The Voss Binary supports three packaging targets:

Target	Method	Description
Linux Static Binary	ELF / static link	Compile VM core + embedded program array into a single statically-linked executable. No runtime dependencies.
Raw `.bin`	Flat binary dump	Dump the code segment directly as a flat binary file. Each 2-byte pair is one Voss instruction word, little-endian.
WASM	WebAssembly module	Same VM logic compiled to WebAssembly. Voss instruction words stored in WASM linear memory. Suitable for browser-based or sandboxed execution.

§13.3 — Operator Signature

The formal operator signature of the Voss Binding:

```
Λ(protagonist_fate: FateLine, external_influence: FateLine) -> MergedFateLine
```

The Λ operator takes two fate lines — the protagonist's current fate and an external influence — and produces a merged fate line that satisfies all seven Λ laws. The merge is deterministic and idempotent for identical inputs.

§13.4 — Side Effects

Every invocation of the Λ operator produces the following side effects:

- `debt.coupling += 5` (standard bind cost; `+= 10` for re-bind)
- Sets `bound_flag = true` on the target
- Modifies future cycle conditions (downstream FLOW decisions are constrained by binding state)
- Introduces a persistent coupling constraint (the target is now fate-linked and cannot silently decouple per Λ_5)

§14 — System Guarantees

The Voss Binary provides four formal guarantees. These are structural properties of the system, not aspirational goals. They hold for any conformant execution as verified by the REP law-proof method (§12).

Guarantee 1: No cycle proceeds without binding.

Every GRE cycle that advances past the Λ boundary must have an active binding. Unbound state cannot cross a cycle boundary.

Guarantee 2: No binding is cost-free.

Every `BIND` operation increases coupling debt. There is no zero-cost binding. The minimum cost is `debt.coupling += 5`.

Guarantee 3: No bound state is consequence-free.

Once a target is bound, its fate flags constrain all subsequent operations on that target. Binding creates obligations that must be honored or explicitly faulted.

Guarantee 4: All future cycles reflect prior coupling.

Coupling state is persistent. Prior bindings, locks, and chain memberships are carried forward into all future cycles. The system has memory; history is not discardable.

> “
>
> *No system evolves in isolation. After stabilization, all state must bind to influence, and that binding must carry cost into all future cycles.*
>
> ”

Appendix A — Quick Reference Card

Voss Binary v1.0 — Quick Reference

Instruction Word Format

```
[15:12] CLASS  |  [11:8] OPCODE  |  [7:0] IMM/FLAGS    (16 bits fixed)
```

Class Map

Hex	Class	Instructions
`0x0`	FLOW	`NEXT`, `ADMIT`, `WAIT`, `HALT`, `FAULT`
`0x1`	BIND/Λ	`BIND`, `UNBIND`, `LOCK`, `RELEASE`, `Λ_CHAIN`
`0x2`	DELTA/Δ	`Δ_ADD`, `Δ_SETTLE`, `Δ_DEGRADE`, `Δ_PROPAGATE`
`0x3`	INVARIANT	`ASSERT_INV`, `GUARD_INV`, `FAIL_INV`
`0x4`	META/GRE	`TRACE`, `GRE_MARK`, `GRE_NEXT_K`

Complete Mnemonic Table

Hex	Mnemonic	Hex	Mnemonic	Hex	Mnemonic
`0x00`	`NEXT`	`0x10`	`BIND`	`0x20`	`Δ_ADD`
`0x01`	`ADMIT`	`0x11`	`UNBIND`	`0x21`	`Δ_SETTLE`
`0x02`	`WAIT`	`0x12`	`LOCK`	`0x22`	`Δ_DEGRADE`
`0x0F`	`HALT/FAULT`	`0x13`	`RELEASE`	`0x23`	`Δ_PROPAGATE`
`0x30`	`ASSERT_INV`	`0x14`	`Λ_CHAIN`	`0x40`	`TRACE`
`0x31`	`GUARD_INV`			`0x41`	`GRE_MARK`
`0x3F`	`FAIL_INV`			`0x42`	`GRE_NEXT_K`

GRE 9-Stage Pipeline

```
S0 FETCH → S1 DECODE → S2 PRECHECK → S3 APPLY_Δ → S4 APPLY_Λ → S5 FLOW_CTRL →
S6 DEGRADE → S7 EMIT → S8 HALT/FAULT
```

Λ Laws (7)

Λ_1	**Origin Law — No spontaneous fate**
Λ_2	Monotone Lock — Irreversible lock
Λ_3	Legal Release — No unlock of locked fate
Λ_4	Coupling — Shared fate in chains
Λ_5	Chain Persistence — No silent decoupling
Λ_6	Status-Fate Coherence
Λ_7	Invariant Alignment

Δ Axioms (5)

Δ_1	**Non-Negativity — `delta[t] ≥ 0`**
Δ_2	Locality — Only Δ-class modifies delta
Δ_3	Conservation — Recomputable transitions
Δ_4	Propagation Integrity — Coupled targets only
Δ_5	Coupling Cost — Binding is never free

© 2026 Jon Halstead. All rights reserved.